

Lecture 5

Probabilistic Analysis and Randomized Algorithms



Slides are based on the textbook and its notes

Introduction

- ❑ This lecture introduces probabilistic analysis and randomized algorithms.
- ❑ *Probabilistic analysis* is the use of probability in the analysis of problems. Most commonly, we use probabilistic analysis to analyze the running time of an algorithm.
- ❑ A *randomized algorithm* is one that “flips coins” as it proceeds, and can make decisions based on the outcomes of these coin flips. If you run a randomized algorithm on the same input over and over, you will see different behavior on different runs.
- ❑ All major programming languages include libraries that make it easy to pick random numbers at will, and randomization is a tool that should be in the toolbox of every serious algorithm designer.

Probability review: Random variables

- **Definition:** A *(discrete) random variable* is a function from the sample space of an experiment to the set of real numbers. That is, a random variable assigns a real number to each possible outcome.
- A random variable is a function. It is not a variable, and it is not random!
- **Definition:** The *distribution* of a random variable X on a sample space S is the set of pairs $(x, \Pr\{X = x\})$ for all $x \in X(S)$, where $f(x) = \Pr\{X = x\}$ is the *probability density function* that X takes the value x .

Probability review: Random variables

Example: Suppose that a coin is flipped three times. Let X be the random variable that equals the number of heads that appear. Then X takes on the following values:

$$\begin{aligned}X(HHH) &= 3, X(TTT) = 0, \\X(HHT) &= X(HTH) = X(THH) = 2, \\X(TTH) &= X(THT) = X(HTT) = 1.\end{aligned}$$

Each of the eight possible outcomes has probability $1/8$. So, the distribution of X is determined by the probabilities $\Pr\{X = 3\} = 1/8$, $\Pr\{X = 2\} = 3/8$, $\Pr\{X = 1\} = 3/8$, and $\Pr\{X = 0\} = 1/8$. Or the set of pairs $(3, 1/8)$, $(2, 3/8)$, $(1, 3/8)$, and $(0, 1/8)$.

Probability review: Random variables

Example: Let X be the sum of the numbers that appear when a pair of dice is rolled. What are the values of this random variable for the 36 possible outcomes (i, j) , where i and j are the numbers that appear on the first die and the second die, respectively, when these two dice are rolled?

$$X((1, 1)) = 2,$$

$$X((1, 2)) = X((2, 1)) = 3,$$

$$X((1, 3)) = X((2, 2)) = X((3, 1)) = 4,$$

$$X((1, 4)) = X((2, 3)) = X((3, 2)) = X((4, 1)) = 5,$$

$$X((1, 5)) = X((2, 4)) = X((3, 3)) = X((4, 2)) = X((5, 1)) = 6,$$

$$X((1, 6)) = X((2, 5)) = X((3, 4)) = X((4, 3)) = X((5, 2)) = X((6, 1)) = 7,$$

$$X((2, 6)) = X((3, 5)) = X((4, 4)) = X((5, 3)) = X((6, 2)) = 8,$$

$$X((3, 6)) = X((4, 5)) = X((5, 4)) = X((6, 3)) = 9,$$

$$X((4, 6)) = X((5, 5)) = X((6, 4)) = 10,$$

$$X((5, 6)) = X((6, 5)) = 11,$$

$$X((6, 6)) = 12.$$

Probability review: Expected value

- ▣ **Definition:** The simplest and most useful summary of the distribution of a random variable is the “average” of the values it takes on. The *expected value* (or *expectation* or *mean*) of a discrete random variable X is

$$E[X] = \sum_x x \cdot \Pr\{X = x\} ,$$

Example: Expected value of a die: Let X be the number that comes up when a fair die is rolled. What is the expected value of X ?

Solution: The random variable X takes the values 1, 2, 3, 4, 5, or 6. Each has probability $1/6$. It follows that

$$E[X] = \frac{1}{6} \cdot 1 + \frac{1}{6} \cdot 2 + \cdots + \frac{1}{6} \cdot 6 = \frac{21}{6} = \frac{7}{2}.$$

Probability review: Expected value

Example: A fair coin is flipped three times. Let S be the sample space of the eight possible outcomes, and let X be the random variable that assigns to an outcome the number of heads in this outcome.

Solution: Because the coin is fair and the flips are independent, the probability of each outcome is $1/8$. Consequently,

$$\begin{aligned} E[X] &= \frac{1}{8}[X(HHH) + X(HHT) + X(HTH) + X(THH) + X(TTH) \\ &\quad + X(THT) + X(HTT) + X(TTT)] \\ &= \frac{1}{8}(3 + 2 + 2 + 2 + 1 + 1 + 1 + 0) = \frac{12}{8} \\ &= \frac{3}{2}. \end{aligned}$$

Probability review: Expected value

Example: What is the expected value of the sum of the numbers that appear when a pair of fair dice is rolled?

Solution: Let X be the random variable equal to the sum of the numbers that appear when a pair of dice is rolled. The range of X of this example is $\{2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12\}$ and we see that

$$\Pr\{X=2\} = \Pr\{X=12\} = 1/36, \Pr\{X=3\} = \Pr\{X=11\} = 2/36 = 1/18,$$

$$\Pr\{X=4\} = \Pr\{X=10\} = 3/36 = 1/12, \Pr\{X=5\} = \Pr\{X=9\} = 4/36 = 1/9,$$

$$\Pr\{X=6\} = \Pr\{X=8\} = 5/36, \Pr\{X=7\} = 6/36 = 1/6.$$

Substituting these values in the formula, we have

$$\begin{aligned} E[X] &= 2 \cdot \frac{1}{36} + 3 \cdot \frac{1}{18} + 4 \cdot \frac{1}{12} + 5 \cdot \frac{1}{9} + 6 \cdot \frac{5}{36} + 7 \cdot \frac{1}{6} \\ &\quad + 8 \cdot \frac{5}{36} + 9 \cdot \frac{1}{9} + 10 \cdot \frac{1}{12} + 11 \cdot \frac{1}{18} + 12 \cdot \frac{1}{36} \\ &= 7. \end{aligned}$$

Probability review: Linearity of expectations

- Expectations are linear: for any two random variables X and Y and any constant a ,

$$E(aX + Y) = aE[X] + E[Y].$$

- The expectation of the sum of n random variables is the sum of their expectations, that is,

$$E[X_1 + X_2 + \dots + X_n] = E[X_1] + E[X_2] + \dots + E[X_n]$$

where X_i , $i = 1, 2, \dots, n$, with n a positive integer, are random variables.

The hiring problem

Scenario

- Suppose that you need to hire a new office assistant.
 - Employment agency sends you one candidate each day.
 - You interview that person and decide either to hire that person (and fire your current office assistant) or not.
 - You always want the best person – if that applicant is better qualified than the current office assistant, you will fire the current office assistant and hire the new applicant.

Goal

- You wish to estimate what that price will be.

The hiring problem: Pseudocode

- Assumes that the candidates are numbered 1 through n and you are able to, after interviewing candidate i , determine whether candidate i is the best candidate you have seen so far.
- To initialize, the procedure creates a dummy candidate, numbered 0, who is less qualified than each of the other candidates. So that the first candidate is always hired.

HIRE-ASSISTANT(n)

```
1  best = 0           // candidate 0 is a least-qualified dummy candidate
2  for  $i = 1$  to  $n$ 
3      interview candidate  $i$ 
4      if candidate  $i$  is better than candidate best
5          best =  $i$ 
6          hire candidate  $i$ 
```

The hiring problem: Costs

- We focus **NOT** on the running time of HIRE-ASSISTANT, but instead on the costs incurred by interviewing and hiring.
- On the surface, analyzing the cost of this algorithm may seem very different from analyzing the running time of an algorithm. However, the analytical techniques used are identical whether we are analyzing cost or running time. In either case, we are counting the number of times certain basic operations are executed.
- The cost to interview a candidate is c_i .
- The cost to hire a candidate is c_h .
- Assume that c_i is much less than c_h .
- Let m be the number of people hired, the total cost associated with this algorithm is $O(c_i n + c_h m)$.

The hiring problem: Best- and Worst- cases

- No matter how many people we hire, we always interview n candidates and thus always incur the cost $c_i n$ associated with interviewing. We therefore concentrate on analyzing $c_h m$, the hiring cost. This quantity varies with each run of the algorithm.
- Best-case cost: $O(c_i n + c_h)$
 - If each candidate is worse than all who came before, we hired once only.
- Worst-case cost: $O(c_i n + c_h m) = O(c_h n)$ since $c_h > c_i$
 - If each candidate is better than all who came before, we actually hire every candidate that we interview. This situation occurs if the candidates come in strictly increasing order of quality, in which case we hire n times.
- Of course, the candidates do not always come in increasing order of quality. Therefore, it is natural to ask what we expect to happen in a typical or average case.

Probabilistic analysis

- ❑ In general, you have no control over the order in which candidates appear. We need to use probabilistic analysis to analyze the hiring cost in procedure HIRE-ASSISTANT.
- ❑ In order to perform a probabilistic analysis, we must use knowledge of, or make assumptions about, the distribution of the inputs.
- ❑ Then we analyze our algorithm, computing an average-case running time, where we take the average over the distribution of the possible inputs. Thus we are, in effect, averaging the running time over all possible inputs. When reporting such a running time, we will refer to it as the *average-case running time*.
- ❑ We must be very careful in deciding on the distribution of inputs.

The hiring problem

□ Average-case cost:

- Assume that the applicants come in a random order, it implies that
- Each permutation of applicants is equally likely.

(Assume that we can compare any two candidates and decide which one is better qualified; that is, there is a total order on the candidates. Thus, we can rank each candidate with a unique number from 1 through n , using $rank(i)$ to denote the rank of applicant i , and adopt the convention that a higher rank corresponds to a better qualified applicant. The ordered list $\langle rank(1), rank(2), \dots, rank(n) \rangle$ is a permutation of the list $\langle 1, 2, \dots, n \rangle$. This list of ranks is equally likely to be any one of the $n!$ permutations of the numbers 1 through n .)

Randomized algorithms

- In order to use probabilistic analysis, we need to know something about the distribution of the inputs. In many cases, we know very little about the input distribution.
- Even if we do know something about the distribution, we may not be able to model this knowledge computationally.
- Yet we often can use probability and randomness as a tool for algorithm design and analysis, by making the behavior of part of the algorithm random.
- In the hiring problem, it may seem as if the candidates are being presented to us in a random order, but we have no way of knowing whether or not they really are.
- Thus, in order to develop a *randomized algorithm* for the hiring problem, we must have greater control over the order in which we interview the candidates. We will change the model₆ slightly.

Randomized algorithms

- The employment agency has n candidates, and they send us a list of the candidates in advance. On each day, we choose, randomly, which candidate to interview.
- Instead of relying on a guess that the candidates come to us in a random order, we have instead gained control of the process and enforced a random order.
- More generally, we call an algorithm *randomized* if its behavior is determined not only by its input but also by values produced by a *random-number generator*. In practice, most programming environments offer a *pseudorandom-number generator*.
- We shall assume that we have at our disposal a random-number generator `RANDOM`. A call to `RANDOM( $a$ ,  $b$ )` returns an integer between a and b , inclusive, with each such integer being equally likely.

Expected running time

- When analyzing the running time of a randomized algorithm, we take the expectation of the running time over the distribution of values returned by the random number generator.
- We distinguish these algorithms from those in which the input is random by referring to the running time of a randomized algorithm as an *expected running time*.
- In general, we discuss the average-case running time when the probability distribution is over the inputs to the algorithm, and we discuss the expected running time when the algorithm itself makes random choices.

Indicator random variables

- In order to analyze many algorithms, including the hiring problem, we use indicator random variables. It provides a convenient method for converting between probabilities and expectations.
- Suppose we are given a sample space S and an event A . Then the *indicator random variable* $I\{A\}$ associated with event A is defined as

$$I\{A\} = \begin{cases} 1 & \text{if } A \text{ occurs ,} \\ 0 & \text{if } A \text{ does not occur .} \end{cases}$$

- **ProofLemma 1:** Given a sample space S and an event A in the sample space S , let $X_A = I\{A\}$. Then $E[X_A] = \Pr\{A\}$.

$$\begin{aligned} E[X_A] &= E[I\{A\}] \\ &= 1 \cdot \Pr\{A\} + 0 \cdot \Pr\{\overline{A}\} \\ &= \Pr\{A\} , \end{aligned}$$

Analysis of the hiring problem

- We now compute the expected number of times that we hire a new office assistant. Let X be the random variable whose value equals the number of times we hire a new office assistant. We could then apply the definition of expected value to obtain

$$E[X] = \sum_{x=1}^n x \Pr\{X = x\} ,$$

- We use indicator random variables to greatly simplify the calculation. We define n variables related to whether or not each particular candidate is hired. In particular, we let X_i be the indicator random variable associated with the event in which the i th candidate is hired. Thus,

$$\begin{aligned} X_i &= I\{\text{candidate } i \text{ is hired}\} \\ &= \begin{cases} 1 & \text{if candidate } i \text{ is hired ,} \\ 0 & \text{if candidate } i \text{ is not hired ,} \end{cases} \end{aligned}$$

and

$$X = X_1 + X_2 + \cdots + X_n .$$

Analysis of the hiring problem

- By Lemma 1, we have that

$$E[X_i] = \Pr\{\text{candidate } i \text{ is hired}\} ,$$

and we must therefore compute the probability that lines 5-6 of HIRE-ASSISTANT are executed.

- Candidate i is hired, in line 6, exactly when candidate i is better than each of candidates 1 through $i - 1$. Because we have assumed that the candidates arrive in a random order, the first i candidates have appeared in a random order. Any one of these first i candidates is equally likely to be the best-qualified so far.
- Candidate i has a probability of $1/i$ of being better qualified than candidates 1 through $i - 1$ and thus a probability of $1/i$ of being hired. By Lemma 1, we conclude that $E[X_i] = 1/i$.

Analysis of the hiring problem

- We can get:

$$\begin{aligned} E[X] &= E\left[\sum_{i=1}^n X_i\right] && \text{(by equation (5.3))} \\ &= \sum_{i=1}^n E[X_i] && \text{(by equation (C.24), linearity of expectation)} \\ &= \sum_{i=1}^n \frac{1}{i} && \text{(by equation (5.4))} \\ &= \ln n + O(1) && \text{(by equation (A.9), the harmonic series) .} \end{aligned}$$

Even though we interview n people, we actually hire only approximately $\ln n$ of them, on average. We summarize this result in the following lemma.

- **Lemma 2:** Assuming that the candidates are presented in a random order, algorithm HIRE-ASSISTANT has an average-case total hiring cost of $O(c_h \ln n)$.
- The average-case hiring cost is significant improvement over the worst-case hiring cost of $O(c_h n)$.

The hiring problem – Randomized algorithm

- In the hiring problem, we assume that all permutations of the input are equally likely and then a probabilistic analysis can guide the development of a randomized problem.
- Instead of assuming a distribution, we impose a distribution. Before running the algorithm, we randomly permute the candidates in order to enforce the property that every permutation is equally likely.
- Although we have modified the algorithm, we still expect to hire a new office assistant approximately $\ln n$ times. But now we expect this to be the case for any input, rather than for inputs drawn from a particular distribution.

The hiring problem – Randomized algorithm

- Let us further explore the distinction between probabilistic analysis and randomized algorithms. In the previous analysis, we claimed that, assuming that the candidates arrive in a random order, the expected number of times we hire a new office assistant is about $\ln n$.
- Note that the algorithm here is deterministic: for any particular input, the number of times a new office assistant is hired is always the same.
- Furthermore, the number of times we hire a new office assistant differs for different inputs, and it depends on the ranks of the various candidates.
- For example, given the following rank lists
 - $A_1 = \langle 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 \rangle$, we hired 10 times.
 - $A_2 = \langle 10, 9, 8, 7, 6, 5, 4, 3, 2, 1 \rangle$, we hired 1 times.
 - $A_3 = \langle 5, 2, 1, 8, 4, 7, 10, 9, 3, 6 \rangle$, we hired 3 times.

The hiring problem – Randomized algorithm

- Consider, on the other hand, the randomized algorithm that first permutes the candidates and then determines the best candidate.
- In this case, we randomize in the algorithm, not in the input distribution. Given a particular input, say A_3 above, we cannot say how many times the maximum is updated, because this quantity differs with each run of the algorithm.
- For this algorithm and many other randomized algorithms, no particular input elicits its worst-case behavior. The randomized algorithm performs badly only if the random-number generator produces an “unlucky” permutation.

The hiring problem – Randomized algorithm

- For the hiring problem, the only change needed in the code is to randomly permute the array.

RANDOMIZED-HIRE-ASSISTANT(n)

- 1 randomly permute the list of candidates
- 2 HIRE-ASSISTANT(n)

- With this simple change, we have created a randomized algorithm whose performance matches that obtained by assuming that the candidates were presented in a random order.
- **Lemma 3:** The expected hiring cost of the procedure RANDOMIZED-HIRE-ASSISTANT is $O(c_h \ln n)$.

The hiring problem – Randomized algorithm

- ❑ Comparing Lemmas 2 and 3 highlights the difference between probabilistic analysis and randomized algorithms.
- ❑ In Lemma 2, we make an assumption about the input. In Lemma 3, we make no such assumption, although randomizing the input takes some additional time.
- ❑ To remain consistent with our terminology, we couched Lemma 2 in terms of the average-case hiring cost and Lemma 3 in terms of the expected hiring cost.

Randomly permuting an array

- **Goal:** Produce a uniform random permutation (each of the $n!$ permutations is equally likely to be produced).
- The following procedure permutes the array $A[1 : n]$ in place.

RANDOMLY-PERMUTE(A, n)

1 **for** $i = 1$ **to** n

2 swap $A[i]$ with $A[\text{RANDOM}(i, n)]$

- **Idea**

- In iteration i , choose $A[i]$ randomly from $A[1 : n]$.
- Will never alter $A[i]$ after iteration i .

- **Time**

- $O(1)$ per iteration $\Rightarrow O(n)$ total.

Reading

- Sections 5.1 ~ 5.3
- Appendix C Counting and Probability